

Core Java Interview Guide

Topic 1: Object-Oriented Programming (OOP)

Set 1 (Questions 1–5)

Q1. What is Object-Oriented Programming (OOP)?

Simple Answer

Object-Oriented Programming (OOP) is a programming paradigm that organizes software into **objects**, which contain both **data (fields)** and **behavior (methods)**. OOP helps build modular, reusable, maintainable, and scalable applications.

Detailed Explanation

Instead of writing everything as functions, OOP groups related data and methods into a single unit called an **object**.

Java is an object-oriented language where almost everything revolves around objects.

The four main principles of OOP are:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

These principles help reduce code duplication and improve maintainability.

Real-Time Example

Think of a **Car**.

A car has:

Data

- Brand
- Color
- Speed

Behavior

- Start()

- Stop()
- Accelerate()

Every car object has its own data but performs similar operations.

Code Example

```
class Car {  
  
    String brand;  
  
    void start() {  
        System.out.println("Car Started");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Car car = new Car();  
  
        car.brand = "Toyota";  
  
        car.start();  
    }  
}
```

Output

Car Started

Interview Tip

If asked "Why OOP?"

Always mention:

- Code Reusability
- Security
- Maintainability
- Scalability

Key Points

- ✓ Java is an Object-Oriented Language.
- ✓ Objects contain data and behavior.
- ✓ OOP improves code reuse and maintenance.
- ✓ Four pillars are Encapsulation, Inheritance, Polymorphism, and Abstraction.

Q2. What is a Class and an Object?

Simple Answer

A **Class** is a blueprint or template used to create objects.

An **Object** is a real-world instance of a class that occupies memory and can access the class members.

Detailed Explanation

A class defines:

- Variables
- Methods
- Constructors

An object is created using the **new** keyword.

Multiple objects can be created from the same class.

Real-Time Example

Blueprint → House

Class → Student

Objects:

- Student1
- Student2
- Student3

Each student has different values but belongs to the same class.

Code Example

```
class Student {  
  
    String name;  
  
    void display() {  
        System.out.println(name);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
        s1.name = "Murali";  
  
        Student s2 = new Student();  
        s2.name = "John";  
  
        s1.display();  
        s2.display();  
    }  
}
```

Output

Murali
John

Interview Tip

Remember:

Class = Blueprint

Object = Real Instance

Interviewers love this simple explanation.

Key Points

✓ Class doesn't occupy memory until an object is created.

✓ Object occupies memory.

✓ One class can create many objects.

Q3. What are the Four Pillars of OOP?

Simple Answer

The four pillars of OOP are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

These principles make software modular, reusable, and easy to maintain.

Detailed Explanation

Encapsulation

Wrapping data and methods together while hiding internal details.

Inheritance

One class acquires the properties and methods of another class.

Polymorphism

One interface, many implementations.

The same method behaves differently based on the object.

Abstraction

Showing only essential information while hiding implementation details.

Real-Time Example

Think about a **Mobile Phone**.

You press the **Call** button.

You don't know how:

- Network connects

- Voice travels
 - Signals are processed
- You only use the functionality.

That's Abstraction.

Code Example

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal Sound");  
    }  
}  
  
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Dog Barks");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Animal animal = new Dog();  
        animal.sound();  
    }  
}
```

Interview Tip

A common interview question is:

"Name the four pillars of OOP."

Answer confidently and explain each with one line.

Key Points

✓ Encapsulation → Data Hiding

✓ Inheritance → Code Reuse

✓ Polymorphism → Multiple Behaviors

✓ Abstraction → Hide Complexity

Q4. What is Encapsulation?

Simple Answer

Encapsulation is the process of **binding data (variables) and methods together** into a single unit (class) while restricting direct access to the data using access modifiers.

It is achieved using **private variables** and **public getter/setter methods**.

Detailed Explanation

Encapsulation protects the internal state of an object.

Instead of allowing direct access to variables, we expose controlled access through methods.

Benefits:

- Data Security
- Better Validation
- Easier Maintenance
- Loose Coupling

Real-Time Example

An **ATM** does not allow you to modify your account balance directly.

You interact only through options like:

- Deposit
- Withdraw
- Check Balance

The balance remains protected internally.

Code Example

```
class Employee {  
  
    private double salary;
```

```
        public void setSalary(double salary) {
            if (salary > 0) {
                this.salary = salary;
            }
        }

        public double getSalary() {
            return salary;
        }
    }

    public class Main {

        public static void main(String[] args) {

            Employee emp = new Employee();

            emp.setSalary(50000);

            System.out.println(emp.getSalary());
        }
    }
```

Output

50000.0

Interview Tip

If someone asks:

"How is Encapsulation achieved in Java?"

Answer:

Using **private variables** with **public getter and setter methods**.

Key Points

- ✓ Provides data hiding.
- ✓ Uses getters and setters.
- ✓ Improves security.

✓ Prevents invalid data modification.

Q5. What is Inheritance?

Simple Answer

Inheritance is a feature where one class acquires the properties and methods of another class.

It promotes **code reusability** and establishes an **IS-A relationship** between classes.

Detailed Explanation

The existing class is called the **Parent (Superclass)**.

The new class is called the **Child (Subclass)**.

Java uses the **extends** keyword for inheritance.

The child class can:

- Access inherited members.
- Add new members.
- Override inherited methods.

Real-Time Example

Consider a company:

Employee contains common details:

- Employee ID
- Name
- Salary

A **Manager** is also an Employee but has additional responsibilities such as approving leave.

Instead of rewriting common properties, the Manager class inherits them from Employee.

Code Example

```
class Employee {  
  
    void work() {  
        System.out.println("Employee is working");  
    }  
}
```

```
class Manager extends Employee {  
  
    void approveLeave() {  
        System.out.println("Manager approved leave");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Manager manager = new Manager();  
  
        manager.work();  
        manager.approveLeave();  
    }  
}
```

Output

```
Employee is working  
Manager approved leave
```

Interview Tip

Whenever asked about inheritance, mention:

- **Code Reusability**
- **IS-A Relationship**
- **Method Overriding** (one of its common uses)

These keywords make your answer stronger.

Key Points

- ✓ Achieves code reuse.
- ✓ Uses the `extends` keyword.
- ✓ Represents an IS-A relationship.
- ✓ Supports method overriding.
- ✓ Java supports single, multilevel, and hierarchical inheritance through classes (multiple inheritance is achieved using interfaces).

Q6. What are the different types of Inheritance in Java?

Simple Answer

Inheritance allows one class to acquire the properties and methods of another class.

Java supports the following types of inheritance:

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance

Java **does not support Multiple Inheritance through classes**, but it can be achieved using interfaces.

Detailed Explanation

1. Single Inheritance

One child class inherits from one parent class.

```
Animal
  ↑
Dog
```

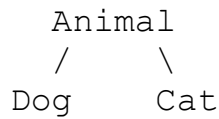
2. Multilevel Inheritance

A child becomes the parent for another class.

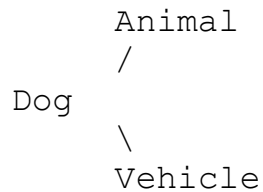
```
Animal
  ↑
Dog
  ↑
Puppy
```

3. Hierarchical Inheritance

Multiple child classes inherit from the same parent.



4. Multiple Inheritance (Not Supported with Classes)



Java doesn't allow this because it creates ambiguity (Diamond Problem).

Instead, Java uses interfaces.

Real-Time Example

A company has:

- Employee
- Manager
- Developer

Both **Manager** and **Developer** inherit common properties like Employee ID, Name, and Salary from the Employee class.

This is an example of **Hierarchical Inheritance**.

Code Example

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Dog dog = new Dog();  
  
        dog.eat();  
        dog.bark();  
    }  
}
```

Output

```
Animal is eating  
Dog is barking
```

Interview Tip

When asked about inheritance types, always mention:

- Java supports **Single**
- **Multilevel**
- **Hierarchical**
- Multiple inheritance is achieved using **Interfaces**, not classes.

Key Points

- ✓ Uses the `extends` keyword.
- ✓ Promotes code reuse.
- ✓ Java doesn't support multiple inheritance with classes.
- ✓ Interfaces solve multiple inheritance.

Q7. Why does Java not support Multiple Inheritance with Classes?

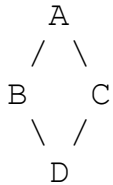
Simple Answer

Java does not support multiple inheritance with classes to avoid the **Diamond Problem**, where a child class inherits the same method from multiple parent classes, causing ambiguity.

Instead, Java supports multiple inheritance using **interfaces**.

Detailed Explanation

Suppose:



Both **B** and **C** override the same method.

When **D** calls that method, Java cannot determine which implementation to execute.

This ambiguity is called the **Diamond Problem**.

Interfaces avoid this issue because Java requires the implementing class to provide the implementation when conflicts occur.

Real-Time Example

Imagine two departments:

HR

```
approveLeave ()
```

Admin

```
approveLeave ()
```

If the Employee class inherits both, which `approveLeave ()` should it execute?

Java avoids this confusion by not allowing multiple inheritance with classes.

Code Example

```
interface Camera {  
  
    void capture();  
}
```

```
interface MusicPlayer {  
  
    void playMusic();  
}  
  
class Mobile implements Camera, MusicPlayer {  
  
    public void capture() {  
        System.out.println("Photo Captured");  
    }  
  
    public void playMusic() {  
        System.out.println("Playing Music");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Mobile mobile = new Mobile();  
  
        mobile.capture();  
        mobile.playMusic();  
    }  
}
```

Interview Tip

If asked:

"Can Java achieve Multiple Inheritance?"

Answer:

Yes, through **Interfaces**, but **not through Classes**.

Key Points

- ✓ Avoids Diamond Problem.
- ✓ Prevents ambiguity.
- ✓ Interfaces support multiple inheritance.
- ✓ Classes do not.

Q8. What is Polymorphism?

Simple Answer

Polymorphism means "**one interface, many forms.**"

It allows the same method to perform different behaviors depending on the object.

There are two types:

- Compile-time Polymorphism (Method Overloading)
- Runtime Polymorphism (Method Overriding)

Detailed Explanation

Polymorphism improves flexibility.

The same method call can execute different implementations.

Example:

```
Animal animal = new Dog();  
animal.sound();
```

Although the reference type is **Animal**, the **Dog** implementation is executed.

Real-Time Example

Think of a **Payment System**.

```
pay()
```

Different payment methods:

- Credit Card
- UPI
- Net Banking

All use the same method name but different implementations.

Code Example

```
class Animal {
```



```
        void sound() {  
            System.out.println("Animal Sound");  
        }  
    }  
  
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Dog Barks");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Animal animal = new Dog();  
  
        animal.sound();  
    }  
}
```

Output

Dog Barks

Interview Tip

Whenever asked about polymorphism, mention both:

- Method Overloading
- Method Overriding

Interviewers almost always ask the follow-up question.

Key Points

- ✓ One interface, many forms.
- ✓ Supports flexibility.
- ✓ Compile-time → Overloading.
- ✓ Runtime → Overriding.

Q9. What is the difference between Method Overloading and Method Overriding?

Simple Answer

Method Overloading	Method Overriding
Same method name with different parameters	Child class provides a new implementation of the parent method
Occurs in the same class	Occurs between parent and child classes
Compile-time polymorphism	Runtime polymorphism

Detailed Explanation

Method Overloading

Same method name but different:

- Number of parameters
- Type of parameters
- Order of parameters

Method Overriding

The child class redefines the parent method using the same:

- Method name
- Return type (or covariant return type)
- Parameters

Real-Time Example

Overloading

```
calculateSalary()  
calculateSalary(int bonus)  
calculateSalary(double incentive)
```

Overriding

```
Employee work()
```

Manager work()

Developer work()

Each employee works differently.

Code Example

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
class Animal {  
  
    void sound() {  
        System.out.println("Animal");  
    }  
}  
  
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Dog");  
    }  
}
```

Interview Tip

A common interview question:

"Can we overload the main() method?"

Answer:

Yes.

"Can we override static methods?"

Answer:

No. Static methods belong to the class and are hidden, not overridden.

Key Points

- ✓ Overloading → Same class.
- ✓ Overriding → Parent and child.
- ✓ Overloading → Compile time.
- ✓ Overriding → Runtime.

Q10. What is Abstraction?

Simple Answer

Abstraction is the process of **hiding implementation details** and showing only the essential functionality to the user.

In Java, abstraction is achieved using:

- Abstract Classes
- Interfaces

Detailed Explanation

Users only interact with what is necessary.

The internal implementation remains hidden.

Benefits:

- Reduces complexity.
- Improves security.
- Makes code easier to maintain.
- Promotes loose coupling.

Real-Time Example

Consider an **ATM**.

You only see:

- Withdraw

- Deposit
- Balance Enquiry

You do not see how:

- Authentication works.
- Database is updated.
- Cash is counted.

These implementation details are hidden.

Code Example

```
abstract class Animal {  
  
    abstract void sound();  
  
    void sleep() {  
        System.out.println("Sleeping...");  
    }  
}  
  
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Dog Barks");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Animal animal = new Dog();  
  
        animal.sound();  
        animal.sleep();  
    }  
}
```

Output

```
Dog Barks  
Sleeping...
```

Interview Tip

A very common follow-up question is:

"How is Abstraction achieved in Java?"

Answer:

Using **Abstract Classes** and **Interfaces**.

Also remember:

- **Abstract Class** can have both abstract and concrete methods.
- **Interface** (Java 8+) can have abstract, default, and static methods.

Q11. What is the difference between an Abstract Class and an Interface?

Simple Answer

Both **Abstract Classes** and **Interfaces** are used to achieve abstraction in Java. An Abstract Class can have both abstract and concrete methods, whereas an Interface primarily defines a contract that implementing classes must follow.

Detailed Explanation

Abstract Class	Interface
Can have abstract and concrete methods	Can have abstract methods, and since Java 8, default and static methods
Can have constructors	Cannot have constructors
Can have instance variables	Can have constants (<code>public static final</code>)
Supports single inheritance	A class can implement multiple interfaces
Uses <code>extends</code>	Uses <code>implements</code>

When to use an Abstract Class?

- When classes share common code and state.
- When you want to provide a partial implementation.

When to use an Interface?

- When unrelated classes need to follow the same contract.
- When multiple inheritance of behavior is required.

Real-Time Example

Abstract Class

An **Employee** class contains common logic such as salary calculation and attendance tracking. Different employee types inherit and customize specific behavior.

Interface

A **Payment** interface can be implemented by:

- CreditCardPayment
- UpiPayment
- NetBankingPayment

Each payment method has its own implementation of the `pay()` method.

Code Example

```
abstract class Employee {  
  
    void login() {  
        System.out.println("Employee Logged In");  
    }  
  
    abstract void work();  
}  
  
class Developer extends Employee {  
  
    @Override  
    void work() {  
        System.out.println("Writing Code");  
    }  
}  
  
interface Payment {
```

```
        void pay();
    }

    class UpiPayment implements Payment {

        @Override
        public void pay() {
            System.out.println("Paid using UPI");
        }
    }
}
```

Interview Tip

A common interview question is:

"Can an Interface have methods with implementation?"

Answer:

Yes. Since Java 8, interfaces can have **default** and **static** methods.

Key Points

- ✓ Both provide abstraction.
- ✓ Abstract Class supports partial implementation.
- ✓ Interface defines a contract.
- ✓ Java supports multiple interfaces but only single class inheritance.

Q12. When should you use an Abstract Class and when should you use an Interface?

Simple Answer

Use an **Abstract Class** when multiple related classes share common state and behavior.

Use an **Interface** when different classes need to follow the same contract, regardless of their relationship.

Detailed Explanation

Use an Abstract Class when:

- Code reuse is needed.
- Common variables exist.
- Common implementation exists.
- There is an **IS-A** relationship.

Use an Interface when:

- Only behavior is common.
- Multiple inheritance is required.
- Loose coupling is desired.
- Different classes implement the same functionality.

Real-Time Example

Abstract Class

```
Employee
  ↑
Developer
Manager
Tester
```

All employees have:

- Employee ID
- Salary
- Login functionality

Interface

```
Payment

Credit Card
UPI
PayPal
```

Every payment method performs payment differently.

Code Example

```
abstract class Vehicle {

    void start() {
```

```
        System.out.println("Vehicle Started");
    }
}

class Car extends Vehicle {

interface Flyable {

    void fly();
}

class Bird implements Flyable {

    public void fly() {
        System.out.println("Bird Flying");
    }
}
```

Interview Tip

A good one-line answer is:

Abstract Class = "What it is."

Interface = "What it can do."

Key Points

- ✓ Abstract Class shares implementation.
- ✓ Interface shares behavior.
- ✓ Interface provides loose coupling.
- ✓ Abstract Class provides code reuse.

Q13. What is the **this** keyword?

Simple Answer

The **this** keyword refers to the **current object** of a class.

It is mainly used to:

- Access instance variables.
- Call another constructor.
- Pass the current object.
- Return the current object.

Detailed Explanation

When a local variable and an instance variable have the same name, Java uses `this` to refer to the instance variable.

Without `this`, Java treats the variable as the local variable.

Real-Time Example

Suppose a Student class has a field named `name`.

When creating a student object, the constructor parameter is also called `name`.

The `this` keyword differentiates them.

Code Example

```
class Student {  
  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(name);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Student s = new Student("Murali");  
  
        s.display();  
    }  
}
```

Output

Murali

Interview Tip

Remember the four common uses of `this`:

- Current object
- Current constructor
- Current object as parameter
- Current object as return value

Key Points

- ✓ Refers to the current object.
- ✓ Resolves variable name conflicts.
- ✓ Used for constructor chaining.
- ✓ Improves code readability.

Q14. What is the **super** keyword?

Simple Answer

The `super` keyword refers to the **immediate parent class object**.

It is used to:

- Access parent variables.
- Call parent methods.
- Invoke the parent constructor.

Detailed Explanation

Whenever a child object is created, the parent constructor executes first.

The `super` keyword allows the child class to access parent members.

Real-Time Example

An **Employee** class has a method called `login()`.

A **Manager** class inherits from `Employee`.

Instead of rewriting the method, the `Manager` can call:

```
super.login();
```

Code Example

```
class Employee {

    Employee() {
        System.out.println("Employee Constructor");
    }

    void work() {
        System.out.println("Employee Working");
    }
}

class Manager extends Employee {

    Manager() {
        super();
        System.out.println("Manager Constructor");
    }

    void display() {
        super.work();
    }
}

public class Main {

    public static void main(String[] args) {

        Manager manager = new Manager();

        manager.display();
    }
}
```

Output

Employee Constructor
Manager Constructor
Employee Working

Interview Tip

Remember:

- `this` → Current object.
- `super` → Parent object.

Interviewers often ask these together.

Key Points

- ✓ Refers to the parent class.
- ✓ Calls parent constructor.
- ✓ Accesses parent methods and variables.
- ✓ Used only in inheritance.

Q15. What is a Constructor? What are its types?

Simple Answer

A **Constructor** is a special method used to initialize an object when it is created.

It has the same name as the class and does not have a return type.

The main types are:

- Default Constructor
- Parameterized Constructor

Detailed Explanation

A constructor is automatically executed whenever an object is created using the `new` keyword.

Default Constructor

A constructor without parameters.

Parameterized Constructor

A constructor with parameters used to initialize object values.

Constructor Overloading

A class can have multiple constructors with different parameter lists.

Real-Time Example

When an employee joins a company, employee details such as ID, Name, and Department are initialized immediately.

A constructor performs this initialization automatically.

Code Example

```
class Employee {  
  
    String name;  
  
    Employee() {  
        name = "Unknown";  
    }  
  
    Employee(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(name);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Employee emp1 = new Employee();  
        Employee emp2 = new Employee("Murali");  
  
        emp1.display();  
        emp2.display();  
    }  
}
```

```
}
```

Output

```
Unknown  
Murali
```

Interview Tip

Interviewers often ask:

"Can a constructor be final, static, or abstract?"

Answer:

No.

Constructors cannot be:

- final
- static
- abstract

They are only used for object initialization.

Key Points

- ✓ Initializes objects.
- ✓ Has the same name as the class.
- ✓ No return type.
- ✓ Supports constructor overloading.
- ✓ Executes automatically during object creation.

Q16. What is the difference between a Constructor and a Method?

Simple Answer

A **Constructor** is used to initialize an object, whereas a **Method** is used to perform an operation or define the behavior of an object.

A constructor is automatically called when an object is created, while a method must be called explicitly.

Detailed Explanation

Constructor	Method
Used to initialize an object	Used to perform operations
Same name as the class	Can have any valid name
No return type	Must have a return type (or void)
Invoked automatically during object creation	Invoked explicitly by calling it
Can be overloaded	Can be overloaded and overridden

Key Difference

- **Constructor** → Initializes the object.
- **Method** → Defines the behavior of the object.

Real-Time Example

When you buy a new **Mobile Phone**:

- The phone is initialized with default settings when it is first turned on (Constructor).
- Making a call, sending a message, or taking a photo are actions (Methods).

Code Example

```
class Employee {  
  
    String name;  
  
    // Constructor  
    Employee(String name) {  
        this.name = name;  
    }  
  
    // Method  
    void display() {  
        System.out.println("Employee Name: " + name);  
    }  
}
```

```
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Employee emp = new Employee("Murali");  
        emp.display();  
    }  
}
```

Output

Employee Name: Murali

Interview Tip

Interviewers often ask:

"Can a constructor return a value?"

Answer:

No. Constructors never have a return type, not even `void`.

Key Points

- ✓ Constructor initializes an object.
- ✓ Method performs operations.
- ✓ Constructor is called automatically.
- ✓ Method is called explicitly.

Q17. What is the **static** keyword? Where is it used?

Simple Answer

The `static` keyword belongs to the **class**, not to individual objects.

A static member is shared by all objects of the class, so it is created only once in memory.

Detailed Explanation

The `static` keyword can be applied to:

- Variables
- Methods
- Blocks
- Nested Classes

Static Variable

Shared among all objects.

Static Method

Can be called without creating an object.

Static Block

Executes once when the class is loaded into memory.

Real-Time Example

In a company, all employees belong to the same company.

Instead of storing the company name in every object, we declare it as a **static variable**, since it is common to all employees.

Code Example

```
class Employee {  
  
    static String company = "Cognizant";  
  
    String name;  
  
    Employee(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(name + " works at " + company);  
    }  
}
```

```
}

public class Main {

    public static void main(String[] args) {

        Employee emp1 = new Employee("Murali");
        Employee emp2 = new Employee("John");

        emp1.display();
        emp2.display();
    }
}
```

Output

```
Murali works at Cognizant
John works at Cognizant
```

Interview Tip

A common interview question:

"Can a static method access non-static variables directly?"

Answer:

No.

A static method can directly access only static members. To access non-static members, it needs an object.

Key Points

- ✓ Belongs to the class.
- ✓ Shared by all objects.
- ✓ Saves memory.
- ✓ Can be used with variables, methods, blocks, and nested classes.

Q18. What is the `final` keyword? How is it used with variables, methods, and classes?

Simple Answer

The `final` keyword is used to **restrict modification**.

It can be applied to:

- Variables
- Methods
- Classes

Detailed Explanation

Final Variable

Once assigned, its value cannot be changed.

```
final int age = 25;
```

Final Method

Cannot be overridden by child classes.

Final Class

Cannot be inherited.

Example:

```
final class Utility {  
}
```

The `String` class is a well-known example of a final class.

Real-Time Example

A company's **GST Number** should never change after registration.

It can be declared as a `final` variable.

Similarly, utility classes or immutable classes are often declared as `final` to prevent inheritance.

Code Example

```
final class Utility {  
  
    void print() {  
        System.out.println("Utility Method");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Utility utility = new Utility();  
  
        utility.print();  
    }  
}
```

Output

Utility Method

Interview Tip

A frequently asked question:

"Can we override a final method?"

Answer:

No.

"Can we inherit a final class?"

Answer:

No.

Key Points

✓ Final variable → Constant value.

✓ Final method → Cannot be overridden.

✓ Final class → Cannot be inherited.

✓ Improves security and immutability.

Q19. What is the difference between Association, Aggregation, and Composition?

Simple Answer

Association, Aggregation, and Composition describe relationships between objects.

- **Association** → Two independent objects are related.
- **Aggregation** → A "Has-A" relationship where the child can exist independently.
- **Composition** → A strong "Has-A" relationship where the child cannot exist without the parent.

Detailed Explanation

Association

Two objects interact but have independent lifecycles.

Example:

Teacher ↔ Student

A teacher can teach many students, and students can exist without a specific teacher.

Aggregation

A weak **Has-A** relationship.

The child object can exist even if the parent object is destroyed.

Example:

Department → Employee

Employees still exist even if a department is removed.

Composition

A strong **Has-A** relationship.

The child object cannot exist without the parent.

Example:

House → Room

If the house is demolished, its rooms no longer exist as part of that house.

Real-Time Example

Association

Customer ↔ Bank

A customer can change banks.

Aggregation

Team → Player

A player can move to another team.

Composition

Car → Engine

The engine is an integral part of the car.

Code Example

```
class Engine {  
    void start() {  
        System.out.println("Engine Started");  
    }  
}  
  
class Car {  
    private Engine engine = new Engine();  
    void startCar() {  
        engine.start();  
    }  
}
```



```
public class Main {  
    public static void main(String[] args) {  
        Car car = new Car();  
        car.startCar();  
    }  
}
```

Output

Engine Started

Interview Tip

Remember this simple rule:

- **Association** → Uses
- **Aggregation** → Has-A (Weak)
- **Composition** → Has-A (Strong)

Key Points

- ✓ Association → Independent relationship.
- ✓ Aggregation → Weak ownership.
- ✓ Composition → Strong ownership.
- ✓ Composition provides tighter coupling than aggregation.

Q20. What are Access Modifiers in Java? Explain **private**, **default**, **protected**, and **public**.

Simple Answer

Access Modifiers control the visibility and accessibility of classes, methods, constructors, and variables.

Java provides four access modifiers:

- `private`
- default (package-private)
- `protected`
- `public`

Detailed Explanation

Modifier	Same Class	Same Package	Subclass (Different Package)	Other Package
<code>private</code>	✓	✗	✗	✗
default	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

When to use each?

- **private** → Hide internal implementation and protect data.
- **default** → Restrict access within the same package.
- **protected** → Allow subclasses to access members.
- **public** → Expose APIs or methods intended for use everywhere.

Real-Time Example

Think of a company building:

- **Private Cabin** → Only the employee can access (`private`).
- **Team Workspace** → Team members can access (default).
- **Manager's Office** → Managers and their teams can access (`protected`).
- **Reception Area** → Everyone can access (`public`).

Code Example

```
class Employee {  
  
    private String salary = "Confidential";  
}
```

```
        public String getSalary() {
            return salary;
        }
    }

    public class Main {

        public static void main(String[] args) {

            Employee emp = new Employee();

            System.out.println(emp.getSalary());

            // System.out.println(emp.salary); // Compilation
Error          }
        }
    }
```

Output

Confidential

Interview Tip

This is one of the most frequently asked Java interview questions.

Memorize the visibility table above. Interviewers often ask candidates to compare **private**, **protected**, and **default** access.

Key Points

- ✓ Controls member visibility.
- ✓ `private` provides the highest level of encapsulation.
- ✓ `public` provides unrestricted access.
- ✓ `protected` supports inheritance.
- ✓ Default access is limited to the same package.